

Chapter 6: Run-Time Environments

编译器必须准确地实现源语言定义中嵌入的抽象。这些抽象通常包括我们在 1.6 节中讨论的概念，例如名称、范围、绑定、数据类型、运算符、过程、参数和控制流结构。

编译器必须与操作系统和其他系统软件合作，以支持目标机器上的这些抽象。为此，编译器创建并管理一个运行时环境，并假定其目标程序正在其中执行。该环境处理各种问题，例如源程序中命名的对象的存储位置的布局 and 分配、目标程序访问变量所使用的机制、过程之间的链接、传递机制参数、操作系统接口、输入/输出设备和其他程序。

本章的两个主题是存储位置的分配以及对变量和数据的访问。我们将详细讨论内存管理，包括栈空间分配和堆管理。

Run-Time Environment

- A compiler must accurately implement the **abstractions** in the source-language definition
 - Names, scopes, bindings, data types, operators, procedures, parameters, flow-of-control constructs...
- To do so, the compiler manages a **run-time environment (运行时刻环境)** and deals with:
 - Layout and allocation of storage locations for data in the source program
 - Mechanisms to access variables
 - Linkages between procedures, the mechanisms for passing parameters
 - Interfaces to the OS, I/O devices, and other programs

• 编译器必须准确实现源语言定义中的抽象概念，包括名称、作用域、绑定、数据类型、运算符、过程、参数、控制流构造等。编译器还必须和操作系统以及其他系统软件协作，在目标机上支持这些抽象概念。

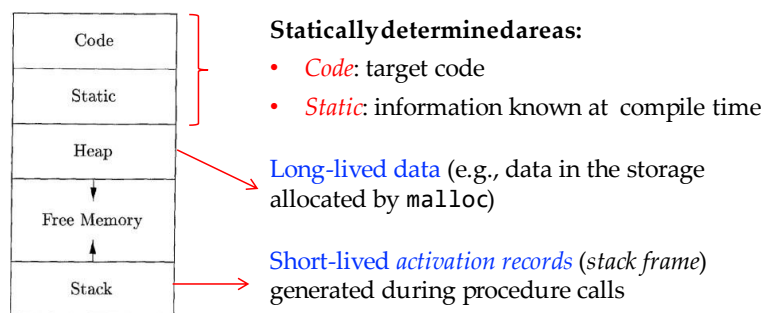
• 为了做到这一点，编译器创建并管理一个运行时环境，它编译得到的目标程序就运行在这个环境中。这个环境处理很多事务，包括内容：

- 为在源程序中命名的对象分配和安排存储位置

- 确定目标程序访问变量时使用的机制
- 过程之间的链接，传递参数的机制
- 与操作系统、I/O 设备和其他程序的接口

Storage Organization

- Typical subdivision of **run-time memory** into **code** and **data areas** (in the logical address space)



从编译器编写者的角度来看，正在执行的目标程序在其自己的逻辑地址空间中运行，每个程序值在该空间中都有一个地址。对这个逻辑地址空间的管理和组织由编译器、操作系统和目标机共同完成。操作系统将逻辑地址映射到物理地址，而物理地址对整个内存空间编址。

一个目标程序在逻辑地址空间的运行时刻映像包含数据区和代码区，如图 7.1 所示。某个语言的 C++ 在某个操作系统（Linux）上的编译器可能会以这种方式划分存储空间。

在本书中，我们假设运行时存储以连续字节块的形式出现，其中字节是内存的最小编址单元。一个字节包含 8 个二进制位，4 个字节组成一个机器字。多字节数据对象总是存储在一段连续的字节中，并把第一个字节作为它的地址。正如第 6 章所讨论的，一个名字所需的存储空间由其类型决定。基本数据类型（例如字符、整数或浮点数）可以存储在整数个字节中。聚合类型（例如数组或结构）的存储必须足够大以存放这个类型的所有分量。

数据对象的存储布局很大程度上受到目标机器的寻址约束的影响。在许多机器上，整数相加指令可能期望整数对齐，即放置在可被 4 整除的地址处。虽然长度为 10 的字符数组（如 C 语言）只需要足够的字节来容纳 10 个字符，但编译器可以分配 12 个字节以获得正确的对齐方式，留下 2 个字节未使用。因为对齐的考虑而产生的闲置空间称为补白。当空间比较紧张时，编译器可能会压缩数据，以消除补白。但是，在运行时可能需要额外的指令来定位被压缩数据，使得机器在操作这些数据时就好像它们是对齐的。

生成的目标代码的大小在编译时就已经固定下来了，因此编译器可以将可执行目标代码放置在静态确定的区域：代码区。这个区通常位于内存的低端。类似地，程序的某些数据对象的大小可以在编译时刻知道，它们可以被放置在另一个称为静态区的区域中，该区域可被静态确定。放置在这个区域的数据对象包括全局常量和编译器生成的数据。如支持垃圾收集的信息等。之所以要将尽可能多的数据对象进行静态分配，是因为这些对象的地址可以被编译到目标代码中。在 Fortran 的早期版本中，所有数据对象都可以静态分配。

为了在运行时最大限度地利用空间，另外两个区域（栈和堆）位于地址空间剩余部分的相对两端。这些区域是动态的；它们的大小可以随着程序的行而改变。这些区域根据需要向对方增长。栈用于存储称为活动记录的数据结构，这些活动记录在过程调用期间生成。

实际上，栈向低地址方向增长，堆向高地址方向增长。然而，在本章和下一章中，我们将假设栈向较高地址方向增长，以便我们可以在所有示例中使用正偏移量。

正如我们将在下一节中看到的，一个活动记录用于在一个过程调用发生时记录有关机器状态的信息，例如程序计数器和机器寄存器的值。当控制从该次调用返回时，相关寄存器的值被恢复，程序计数器被设置成指向紧跟在这次调用后的点，然后调用过程的活动就可以重新开始。如果一个数据对象的生命周期包含在一次活动的生命周期中，那么该数据对象可以与其他关于该活动的信息一起被分配在堆栈上。

许多编程语言允许程序员通过程序控制人工分配和释放数据对象。例如，C 具有函数 `malloc` 和 `free`，可用于获取和释放任意存储块。堆用于管理这种具有长生命周期的数据。

7.4 节将讨论可用于维护堆的各种内存管理算法。

静态确定的区域包括：

Code（代码）： 目标代码，即在编译时已知的信息，用于生成最终可执行程序指令序列。

Static (静态): 编译时已知的信息, 通常是与全局变量、静态变量以及其他在程序生命周期内保持不变的实体有关的信息。

在这两个静态确定的区域之外, 还有一些涉及动态分配和运行时的区域, 例如:

Long-lived data (长寿命周期的数据): 例如, 由 malloc 分配的存储空间中的数据。这些数据的生命周期可能会跨足过程调用或程序执行的多个阶段。

Short-lived activation records (短寿命周期的激活记录, 也称为栈帧): 在过程调用期间生成的, 用于存储有关该过程执行的信息, 例如局部变量、返回地址等。这些记录通常存储在程序执行的堆栈上, 其生命周期仅限于相应的过程调用。

Static vs. Dynamic Storage Allocation

- **Static**: the storage-allocation decision can be made by the compiler by looking only at the program text
 - Global constants, global variables
- **Dynamic**: a decision can be made only while the program is running
 - **Stack storage**: The space for names local to a procedure is allocated on a stack. The lifetime of the data is the same as that of the called procedure
 - **Heap storage**: Hold data that may outlive the call to the procedure that created it
 - **Manual memory deallocation** (手工回收内存, e.g., by calling the function free)
 - **Automatic memory deallocation** (a.k.a., garbage collection, 垃圾回收)

运行时环境中数据的布局和内存位置的分配是存储管理中的关键问题。这些问题很棘手, 因为程序文本中的同一个名称在运行时可能指向不同的存储位置。静态和动态这两个形容词分别表示编译时和运行时。

如果编译器可只需通过观察程序的文本而不需要观察该程序在运行时做了什么, 就能做出存储分配决策, 那么我们就说这个存储分配决策是静态的。这包括全局常量和全局变量。

相反，如果只能在程序运行时做出决定，则该决定就是动态的。许多编译器使用以下两种策略的某种组合来进行动态存储分配：

- Stack storage（栈存储）：用于存储局部于一个过程的名称，这些数据的空间是在一个栈上分配的。数据的生命周期与调用的过程相同。
- Heap storage（堆存储）：用于保存可能超越创建它的过程调用的数据。这些数据的生命周期可能比创建它的过程更长。堆是虚拟内存的一个区域，它允许对象或其他数据元素在被创建时获得存储空间，并在数据变得无效时释放该存储空间。

Activation of Procedures

- Procedure calls, or *activation of procedures*, nest in time
 - If a procedure P calls Q , the callee Q returns before its caller P
 - This makes it possible to use stack (*first in/called, last out/return*) to manage the run-time memory used by procedure calls

如果过程调用或过程活动在时间上不是嵌套的，则堆栈分配就不可行。下面的示例说明了过程调用的嵌套情形。

- 如果过程 P 调用了过程 Q ，被调用者 Q 在其调用者 P 之前返回。
- 这使得可以使用栈（先进/调用，后出/返回）来管理过程调用使用的运行时内存。

```

int a[11];
void readArray() { /* Reads 9 integers into a[1], ..., a[9]. */
    int i;
    ...
}
int partition(int m, int n) {
    /* Picks a separator value v, and partitions a[m..n] so that
       a[m..p-1] are less than v, a[p] = v, and a[p+1..n] are
       equal to or greater than v. Returns p. */
    ...
}
void quicksort(int m, int n) {
    int i;
    if (n > m) {
        i = partition(m, n);
        quicksort(m, i-1);
        quicksort(i+1, n);
    }
}
main() {
    readArray();
    a[0] = -9999;
    a[10] = 9999;
    quicksort(1,9);
}

```

Possible activations:

```

enter main()
  enter readArray()
  leave readArray()
  enter quicksort(1,9)
    enter partition(1,9)
    leave partition(1,9)
    enter quicksort(1,3)
    ...
    leave quicksort(1,3)
    enter quicksort(5,9)
    ...
    leave quicksort(5,9)
  leave quicksort(1,9)
leave main()

```

图 7.2 包含一个程序概要，该程序将 9 个整数读入数组 `a` 并使用递归快速排序算法对它们进行排序。

程序主函数有三个任务。它调用 `readArray`，设置上下限值，然后对整个数组调用快速排序 `quicksort`。图 7.3 显示了程序执行可能产生的一系列调用。在此执行中，对 `partition(1; 9)` 的调用返回 4，因此 `a[1]` 到 `a[3]` 保存小于其所选分隔符值 `v` 的元素，而较大的元素位于 `a[5]` 到 `a[9]` 中。

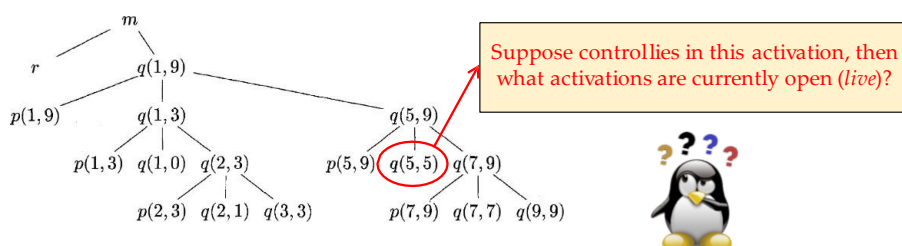
在此示例中，过程活动在时间上是嵌套，在一般情况下也这样。如果过程 `p` 的一个活动调用了过程 `q`，则 `q` 的该次活动必定在 `p` 的活动结束之前结束。常见的情况有以下三种：

1. `q` 的该次活动正常终止。那么，基本上在任何语言中，控制流从 `p` 中调用 `q` 的点之后继续。
2. `q` 的该次活动（或者 `q` 调用的某个过程）直接或间接地中止了，也就是说执行不可能继续下去。在这种情况下，`p` 与 `q` 同时结束。
3. `q` 的该次活动由于 `q` 无法处理的异常而终止。过程 `p` 可能会处理这个异常。此时 `q` 的活动已经终止，而 `p` 的活动继续，尽管 `p` 的活动不一定从调用 `q`

的点开始。如果 p 无法处理异常，则 p 的活动与 q 的活动同时终止。一般来说，某个过程的尚未结束的活动将处理这个异常。

Activation Trees (活动树)

- We can represent the activations of procedures during the running of an entire program by a tree, called *activation tree*
 - Each **node** corresponds to one activation
 - The **root** is the activation of the “main” procedure
- Two interesting properties:**
 - The sequence of procedure calls corresponds to a preorder traversal
 - The sequence of returns corresponds to a postorder traversal



Fall 2024

Compilers Chapter 6

9

因此，我们可以用一棵树来表示整个程序运行期间过程的活动，称为活动树。

每个节点对应于一个活动，而根是启动程序执行的 “main” 过程的活动。

在过程 p 的某个活动的结点上，其子结点对应于被 p 的这次活动调用的各个过程的活动。我们按照这些活动被调用的顺序，从左到右显示这些活动。请注意，一个子节点必须在其右侧兄弟节点的活动开始之前结束。

图 7.3 中给出了一个调用和返回序列，而图 7.4 中则显示了一棵完成这个调用/返回序列的可能的活动树。各个函数由其名称的第一个字母表示。请记住，这棵树只是一种可能性，因为后续调用的参数以及沿各个分支的调用次数都受到 `partition` 的返回的值的影

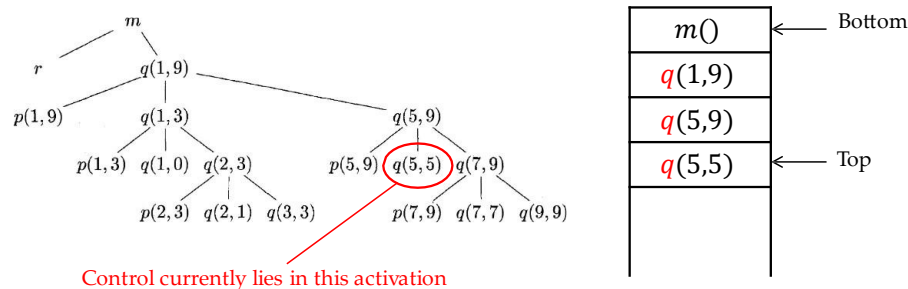
响。在活动树和程序行为之间存在下列多种有用的对应关系，正因为这些关系使我们可以使用运行时刻栈

- 过程调用的顺序对应于活动树的先序遍历。
- 过程返回的顺序对应于活动树的后序遍历。

3. 假设控制位于某个过程的特定活动中，且该过程活动对应于活动树上的某个结点 N 。那么当前尚未结束的活动（即活跃的）活动就是结点 N 及其祖先结点对应的活动。这些活动被调用的顺序是它们从根结点到 N 的路径上的出现顺序。这些活动将按照该顺序反序返回。

Activation Record (活动记录)

- Procedure calls and returns are usually managed by a run-time stack called the *control stack* (or *call stack*)
- Each live activation has an *activation record* (or *stack frame*) on the control stack

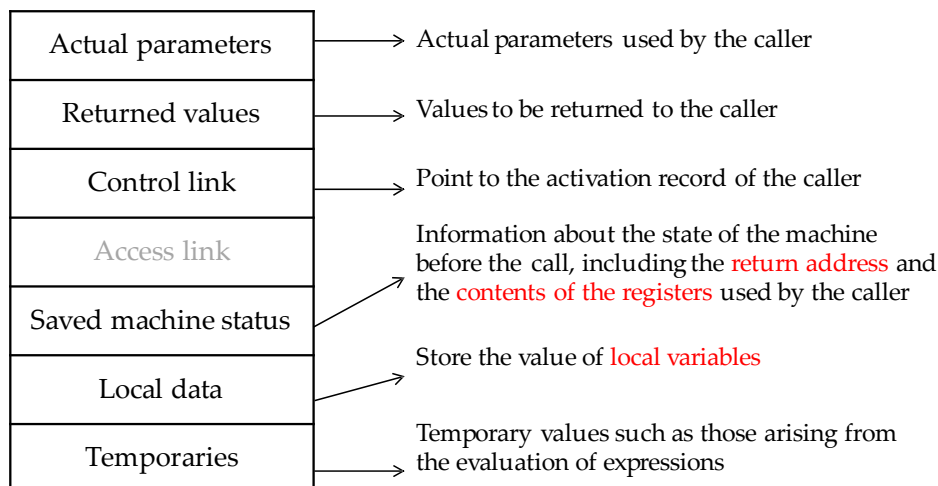


过程调用和返回通常由称为控制栈的运行栈管理。每个活跃的活动在控制栈上都有一个活动记录（有时称为帧）。活动树的根位于栈底部，栈上的全部活动记录的序列对应于活动树中到达当前控制所在活动结点的路径。程序控制所在的活动记录位于栈顶。

如果当前控制位于图 7.4 树的活动 $q(5; 5)$ 上中，则 $q(5; 5)$ 的活动记录位于控制栈的顶部。紧跟在下方的是 $q(5; 9)$ 的活动记录，即树中 $q(5; 5)$ 的父结点。再下面是活动记录 $q(1; 9)$ ，栈的底端主函数 m 的活动记录，也就是活动树的根。

我们通常会绘制控制栈，使堆栈的底部高于顶部，因此在一个活动记录中出现在页面最下方的元素实际上最靠近栈顶。

What's in An Activation Record?



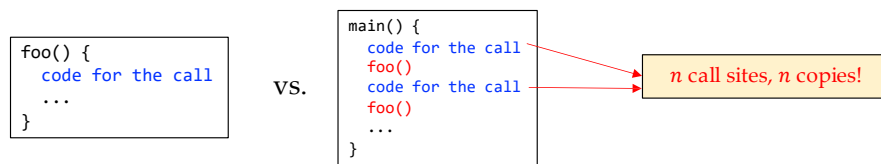
根据所实现的语言的不同，其活动记录的内容也有所不同。以下是活动记录中可能出现的数据类型列表（有关这些元素及它们之间的可能顺序，请参见图 7.5）

1. 临时值，例如由表达式求值过程中产生的中间结果无法保存在寄存器中，就会生成这些临时值。
2. 对应于这个活动记录的过程的局部数据。
3. 保存的机器状态，其中包含对此过程的此次调用之前的机器状态信息。该信息通常包括返回地址（程序计数器的值，被调用的过程必须返回到该值所指的位置）以及一些寄存器中的内容（调用过程会使用这些内容，被调用过程必须在返回时恢复这些寄存器的内容）。
4. 一个“访问链”。当被调用过程需要其他地方（例如，在另一个活动记录中）的某个数据时需要使用访问链进行定位。访问链接在第 7.3.5 节中讨论。
5. 一个控制链，指向调用者的活动记录。
6. 当被调用函数有返回值时，要有一个用于存放这个返回值的的空间。并非所有被调用的过程都会有返回值，即使有，我们也可能更愿意将该值放在一个寄存器中以提高效率。

7. 调用过程使用的实际参数。这些值通常尽可能地放寄存器中，而不是放在活动记录中，因为放在寄存器中会得到更好地效率。然而，我们仍然为它们预留了相应的空间，使得我们的活动记录具有完全的通用性。

Calling Sequences (方法调用代码序列)

- *Calling sequences*, consisting of code that (1) allocates an activation record on the stack and (2) enters information into its fields
- A *return sequence* (返回代码序列) restores the state of the machine so that the caller can continue its execution after the call
- The code in a calling sequence is **divided between the caller and the callee**
 - There is **no exact division**; the source language, target machine, and the OS may impose requirements that affect the solution
 - A general rule is to **put as much code as possible into the callee**



过程调用是通过所谓的调用序列来实现的。

调用序列：为一个活动记录在栈中分配空间，并在此记录的字段中填写信息。

返回序列：恢复机器状态，使得调用过程能够在调用结束之后继续执行。

即使对于同一种语言，不同实现中的调用序列和活动记录的布局也可能有很大差异。

调用序列中的代码通常被分割到调用过程（调用者）和被调用过程（被调用者）中。在分割运行时刻任务时，调用者和被调用者之间不存在明确界限。源语言、目标机器和操作系统会提出某些要求，使得能够选择出一种较好的解决方案。一般来说，如果一个过程在 n 个不同的点上被调用，分配给调用者的那部分调用代码序列会被生成 n 次。然而，分配给被调用者的部分仅被生成一次。因此，最好将调用序列中尽可能多的部分放入被调用者中——能够根据被调用者的信息定的部分都应该放到被调用者中。不过，我们将看到被调用者不可能知道所有的事情。

Calling/Return Sequences' Job

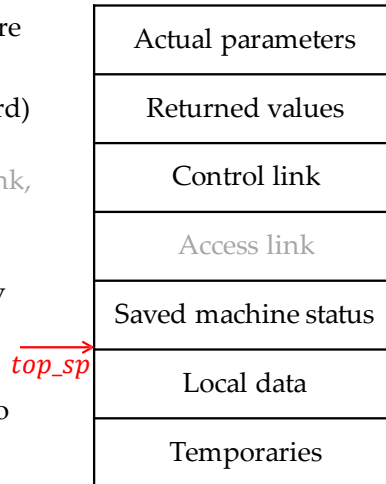
- From the **data** perspective
 - Correctly pass arguments to the callee
 - Correctly pass the return values to the caller
- From the **control** perspective
 - Correctly transfer the control to the first instruction of the callee
 - Correctly transfer the control back to the caller so that it can continue with the instruction immediately after the procedure-call statement

- 从数据角度看
 - 正确地向被调用者传递参数
 - 正确地将返回值传递给调用者
- 从控制角度看
 - 将控制权正确转移到被调用者的第一条指令上
 - 正确地将控制权转回调用程序，以便调用程序在调用语句执行完毕后立即继续执行指令

Layout of Activation Records

General Principles

- Values passed between caller and callee are put at the beginning of the callee's activation record (next to the caller's record)
- Fixed-length items (control link, access link, saved machine status) are in the middle
- Items whose size may not be known early enough are placed at the end
- The top-of-stack (*top_sp*) pointer points to the end of the fixed-length fields



在设计调用序列和活动记录的布局时，可以使用下列原则：

1. 调用者和被调用者之间传递的值通常放在被调用者活动记录的开始位置，因此它们尽可能接近调用者的活动记录。这样做的动机是调用者可以计算该次调用的实在参数的值并将它们放置在自身活动记录的顶部，而不必创建整个被调用者的活动记录，甚至不需要知道该记录的布局。此外，它还使得语言可以使用参数个数或类型可变的过程，例如 C 的 `printf` 函数。被调用者知道应该把返回值放置在相对于其自己的活动记录的哪个位置。同时，不管有多少个参数，它们都将在栈中按顺序地出现在该位置的下方。

2. 固定长度的项被放在中间位置。从图 7.5 中，这些项通常包括控制链、访问链和机器状态字段。如果每次调用中都保存的机器状态的成分相同，那么可以使用同一段代码来保存和恢复每次调用的数据。此外，如果我们机器的状态信息标准化，那么如果发生错误，诸如调试器等这样的程序可以更容易地将栈中的内容解码。

3. 那些在早期无法知道大小的项被放置在活动记录的末尾。大多数局部变量都有固定的长度，编译器通过检查变量的类型就可以确定其长度。然而，有些局部变量的大小只有在程序执行时才能确定；最常见的示例是动态数组，数组大小根据被调用者的某个参数决定。此外，临时量所需的空间的大小通常取决于代码生成阶段能够将多少临时变量放在寄

寄存器中。因此，虽然编译器最终可以知道临时变量所需的空間，但在刚开始生成中间代码时可能并不知道該空間的大小。

4. 我们必须小心确定栈顶指针所指的位置。一种常见的方法是让这个指针指向活动记录中固定长度字段的末尾。这样固定长度的数据就可以通过固定的相对于栈顶指针的偏移量来访问，而中间代码生成器知道这些偏移量。使用这种方法的后果是活动记录中的可变长度字段实际上位于堆栈之上。它们的偏移量需要在运行时计算，但是它们仍然可以基于栈顶指针进行访问，但偏移量为正。

Steps of A Calling Sequence

1. The caller evaluates the **actual parameters**
2. The caller stores a return address and the old value of ***top_sp*** into the callee's activation record
3. The caller increments ***top_sp*** accordingly
4. The callee saves the **register values** and other status info
5. The callee initializes its **local data** and begins execution

图 7.7 给出了调用者和被调用者如何合作管理堆栈的示例。寄存器 `top_sp` 指向当前的顶层活动记录中 机器状态字段的末尾。调用者知道这个位于被调用者的活动记录中的位置。因此，调用者可以负责在控制转向被调用者之前设定 `top sp` 的值。这个调用代码序列以及它在调用者和被调用者之间的划分如下：

1. 调用者计算实在参数的值。
2. 调用者将返回地址和原来的`top_sp`值存放到被调用者的活动记录中。

3. 调用者增加`top_sp`的值（为将来的栈操作做准备），使之指向图 7-7 所示的位置。也就是说，`top_sp`超过了调用者的局部数据和临时变量以及被调用者的参数和机器状态字段。

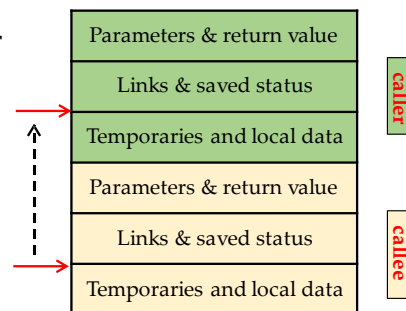
4. 被调用者保存寄存器值和其他状态信息。

5. 被调用者初始化其局部数据并开始执行。

Steps of A Return Sequence

- The callee places the **return value** next to the actual parameters fields in its activation record
- Using information in the machine-status field, the callee restores **`top_sp`*** and other **registers**
- Go to the **return address** set by the caller

* Although `top_sp` has been decremented, the caller knows where to find the return value and use it (recall that caller places the actual parameters)



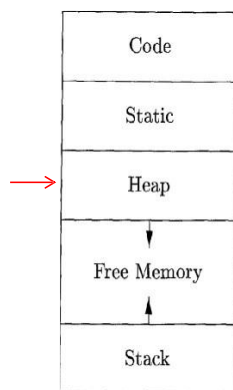
一个与此匹配的返回代码序列：

1. 被调用者将返回值放在参数相邻的位置，如图 7.5 所示。
2. 使用机器状态字段中的信息，被调用者恢复 `top_sp` 和其他寄存器，然后跳转到由调用者放置在状态字段中的返回地址。
3. 尽管 `top_sp` 已经被减小，但调用者知道返回值相对于当前 `top_sp` 值的位置。因此，调用者可以使用那个返回值。

上述调用和返回序列支持使用不同数量的参数来调用同一个被调用程序（例如，如 C 的 `printf` 函数）。请注意，在编译时，调用者的目标代码知道它向被调用中提供的参数的数量和类型。因此调用者知道参数区域的大小。然而，被调用者的目标代码必须还能处理其他调用，因此它要等到被调用时再检查相应的参数字段。使用图 7.7 的组织方法，描述

参数的信息必定放在状态字段的相邻位置，因此被调用者可以找到这个信息。例如，在 C 的 `printf` 函数中，第一个参数描述了其余的参数，因此一旦找到第一个参数，被调用者就可以找到任何其他参数。

Heap (堆)



- The *heap* is used for data that *lives indefinitely*, or until the program explicitly deletes it
- Many languages enable us to create objects/data whose existence is not tied to the procedure activation that creates them
 - In Java, the objects created with *new* can exist long after the procedure that created them is gone
 - In C, the memory space allocated with *malloc* is accessible until the *free* operation is applied

堆是存储空间的一部分，它被用于存储那些生命周期不确定，或将生存到被程序显式删除为止的数据。虽然局部变量会在它们所属的过程结束时变得不可访问，但许多语言支持创建对象或其他数据，它们的存在与否和创建它们的过程活动无关。

例如，C++ 和 Java 都为程序员提供了 `new` 语句，该语句创建的对象（或指向对象的指针）可以在过程间进行传递。因此，这些对象在创建它们的过程结束后仍然可以存在很长一段时间。此类对象存储在堆上。

在 C 语言中，使用 `malloc` 分配的内存空间在执行 `free` 操作之前都是可以访问的

The Memory Manager

- Memory manager **allocates and deallocates space within the heap**
 - It serves as an **interface** between application programs and the OS
- The mechanism depends on languages
 - For C/C++, developers need to deallocate memory manually (using **free** or **delete**). The memory manager is responsible for implementing deallocation
 - For Java, garbage collector (a subsystem of memory manager) finds spaces within the heap that are not longer used and reallocates such spaces to hold other data items

在本节中，我们将讨论内存管理器，即在堆内分配和释放空间的子系统；它充当应用程序和操作系统之间的接口。

对于像 C 或 C++ 这样需要手动回收存储块的语言（即通过程序的显式语句，例如 `free` 或 `delete` 进行回收）而言，内存管理器还负责实现空间回收。

在第 7.5 节中，我们讨论垃圾回收，即在堆中查找程序不再使用的空间的过程，因此可以重新分配这些空间来容纳其他数据项。对于像 Java 这样的语言，内存的回收是垃圾收集器完成的。当需要进行垃圾回收时，垃圾收集器是内存管理器的一个重要子系统。

Basic Functions

- **Allocation (分配内存):** Provide contiguous heap memory when a program requests memory for a variable or object
 - If possible, it satisfies the request [using free space in the heap](#)
 - Otherwise, it seeks to increase the heap storage space by [getting consecutive bytes of virtual memory](#) from the OS
- **Deallocation (回收内存):** Return deallocated space to the pool of free space for reuse
 - Typically, memory managers do not return memory to the OS, even if the program's heap usage drops

内存管理器始终跟踪堆存储中的所有空闲空间。它具有两个基本功能：

- 分配。当程序为变量或对象请求内存时，内存管理器会生成一段连续的具有被请求大小的堆空间。如果有可能，它使用堆中的空闲空间来满足分配请求；如果没有所需大小的空间块可供分配，它试图从操作系统中获取连续的虚拟内存来增加堆的存储空间。如果空间耗尽，内存管理器会将该信息传递回应用程序。
- 回收。内存管理器将释放的空间返回到空闲空间的缓冲池中，以便它可以重用该空间来满足其他分配请求。即使当这个程序的不再需要那么多的堆空间时，内存管理器通常也不会将内存返回给操作系统。

如果下面的 (a)、(b) 两个条件都成立，内存管理会更简单：

- (a) 所有分配请求都是要求相同大小的存储块，
- (b) 存储空间按照可预测地方式释放。例如，先分配先回收。

有一些语言，例如 Lisp，条件 (a) 成立；纯 Lisp 仅使用一种数据元素——一个双指针单元，所有数据结构都是在该元素的基础上构建的。条件 (b) 在某些情况下也成立，最常见的是可以在运行时刻栈上分配的数据。然而，在大多数语言中，(a) 和 (b) 都不普遍成

立。相反，我们需要为不同大小的数据元素分配空间，并且没有好的方法来预测所有已分配对象的生命周期。

因此，内存管理器必须准备好以任何顺序来处理任何大小的空间分配和回收请求，这些请求小到一个字节大到该程序的整个地址空间

Desirable Properties

- **Space efficiency (空间效率):** Minimize the total heap space needed by a program, i.e., minimizing “**fragmentation**” (碎片化)
- **Program efficiency (程序效率):** Make better use of the memory hierarchy to allow programs to run faster*
- **Low overhead (低开销):** Minimize the execution time spent on memory allocation/deallocation

* The time taken to execute an instruction can vary widely depending on where objects are placed in memory

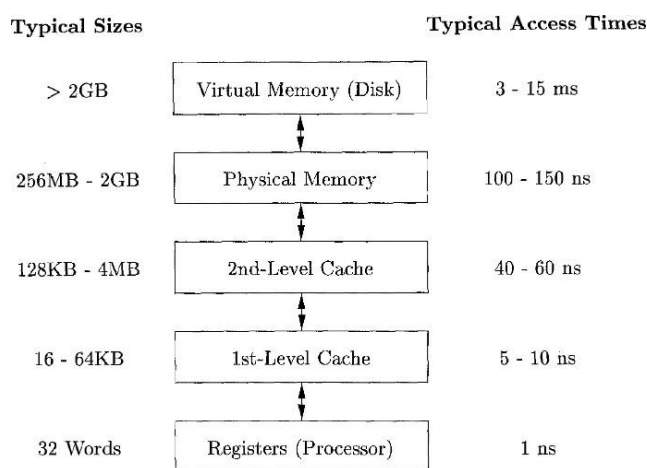
以下是我们希望内存管理器具有的属性：

- 空间效率。内存管理器应该最大限度地减少程序所需的总堆区空间。这样做就可以在一个固定大小的虚拟地址空间中运行更大的程序。空间效率是通过使存储碎片达到最少而得到的。” 在第 7.4.4 节中讨论。

- 程序效率。内存管理器应该充分利用存储子系统，使程序运行得更快。正如我们将在第 7.4.2 节中看到的，根据数据对象在内存中所处的不同位置，执行一条指令所花费的时间可能会有很大差异。幸运的是，程序往往会表现出“局部性”，这是第 7.4.3 节中讨论的一种现象，它指的是通常的程序访问内存时具有的非随机性聚焦的特性。通过关注对象在内存中的放置方法，内存管理器可以更好地利用空间，并有望使程序运行得更快。

- 低开销。由于内存分配和回收是许多程序中的频繁操作，因此这些操作尽可能高效非常重要。也就是说，我们希望最小化开销，即花费在分配和回收上的执行时间在总运行时间所占的比例。请注意，分配开销主要由小型请求决定；管理大型对象的开销相对不太重要，因为它通常会在它上执行大量的计算，这个开销被分摊了。

Memory Hierarchy



内存管理和编译器优化必须在充分了解存储行为方式的情况下进行。现代机器的设计使程序员无需关心内存子系统的细节就可以编写正确的程序。然而，程序的效率不仅取决于执行的指令数量，还取决于执行每条指令所需的时间。不同情况下执行一条指令所花费的时间可能会有很大差异，因为访问不同的存储区域所花费的时间可能从纳秒到毫秒不等。因此，数据密集型程序可以从充分利用内存子系统的优化中获益匪浅。正如我们将在第 7.4.3 节中看到的，它们可以利用“局部性”现象——典型程序的非随机行为。

内存访问时间的巨大差异是由于硬件技术的根本限制造成的；我们可以构建小而快的存储，或者大而慢的存储，但不能构建既大又快的存储。即使是现在也不可能构建具有纳秒访问时间的千兆字节存储，而纳秒级正是高性能处理器的运行速度。因此，几乎所有现代计算机都将其存储安排为内存层次结构。如图 7.16 所示，内存层次结构由一系列存储

元素组成，较小且较快的存储元素距离处理器“较近”，而较大且较慢的存储元素则距离处理器较远。

通常，处理器具有少量寄存器，寄存器中的内容由软件控制。其次，它具有一级或多级高速缓存，这些高速缓存通常使用静态 RAM 制成，大小为千字节到几兆字节。层次结构的下一层是物理（主）内存，由数百兆字节或千兆字节的动态 RAM 组成。然后，物理内存由下一层的虚拟内存提供支持，虚拟内存是由千兆字节的磁盘实现的。在进行内存访问时，机器首先在最近（最底层）的存储中查找数据，如果数据不存在，则到上一层中寻找，依此类推。

寄存器是稀缺的，因此寄存器的使用是针对特定应用程序定制的，并由编译器生成的代码进行管理。存储层次结构的所有其他层都是自动管理的；这样，不仅简化了编程任务，而且同一个程序可以在具有不同存储配置的机器上高效地运行。每次访问内存时，机器都会从最低层开始逐层搜索每一层的存储，直到找到数据。高速缓存是完全通过硬件进行管理的，以便跟上相对较快的 RAM 访问时间。因为磁盘访问速度相对较慢，虚拟内存是由操作系统进行管理的，辅以称为“转换旁视缓冲”的硬件结构。

数据以连续存储块进行传输。为了分摊访问开销，较大的块用于层次结构中较慢的级别。在主内存和高速缓存之间的数据是按照被称为高速缓存线的块进行传输的，高速缓存线的长度通常为 32 到 256 字节之间。在虚拟内存（磁盘）和主内存之间的数据以被称为页的内存形式进行传输，大小通常在 4K 到 64K 字节之间。

Program Locality (程序局部性)

- Most programs exhibit a high degree of locality:
 - **Temporal locality (时间局部性):** the memory locations accessed are likely to be accessed again within a short period of time
 - **Spatial locality (空间局部性):** memory locations close to the locations accessed are likely also to be accessed within a short period of time
- Programs spend 90% of their time executing 10% of the code (80/20 or Pareto's Principle in programming)

Locality allows us to take advantage of the memory hierarchy to lower the average memory-access time: place the most common instructions and data in the fast-but-small storage, while leaving the rest in the slow-but-large storage

大多数程序都表现出高度的局部性；也就是说，它们将大部分时间花在执行相对较小的代码部分并仅接触一小部分数据。

如果一个程序访问的内存位置很可能在短时间内被再次访问，我们就说该程序具有**时间局部性**。

如果靠近被访问位置的内存位置也可能在短时间内被访问，我们就说程序具有**空间局部性**。

传统观点认为，程序花费 90% 的时间执行 10% 的代码（80/20 或编程中的帕累托原则）。原因如下：

程序经常包含许多从未执行过的指令。使用组件和库构建的程序仅使用了它们所提供功能的一小部分。此外，随着需求的变化和程序的演化，遗留系统通常包含许多不再使用的指令。

在程序的一次典型运行中，可能被调用的代码中只有一小部分会被实际执行。例如，虽然处理非法输入和异常情况的指令虽然对程序的正确性至关重要，但它们在某次运行中很少被调用。

通常的程序往往大部分时间都花费在执行程序中的最内层循环和最紧凑的递归循环上。

局部性使我们能够利用现代计算机的存储层次结构，如图 7.16 所示。通过将最常用的指令和数据放在快速但较小的存储中，而将其余的留在缓慢但较大的存储中，我们可以显著降低程序的平均内存访问时间。

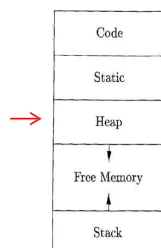
已经发现，许多程序在访问指令和数据的方式上既表现出时间局部性，又表现出空间局部性。然而，数据访问模式通常比指令访问模式表现出更大的多样性。将最近使用的数据放在最快的存储层次中等策略对于常见程序来说效果很好，但对于某些数据密集型程序可能效果不佳。例如，循环遍历非常大的数组。

仅通过查看代码，我们通常无法判断代码的哪些部分将被大量使用，尤其是对于特定输入这一点则更加困难。即使我们知道哪些指令会被频繁执行，最快的高速缓存通常也不足以同时容纳所有指令。因此，我们必须动态调整最快存储的内容，并用它来保存在可能很快会被使用的指令。

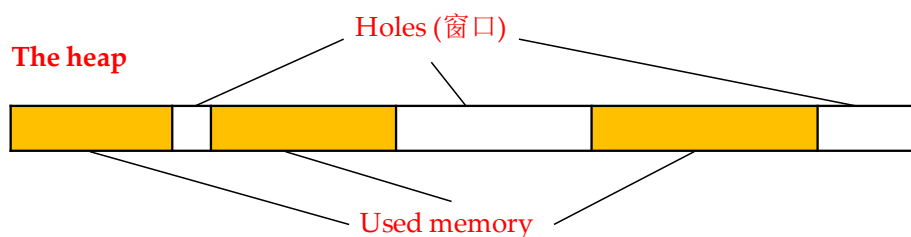
将最近使用的指令保留在高速缓存中的策略往往效果很好；换句话说，过去的情况通常可以很好地预测未来的内存使用情况。当执行一条新指令时，很有可能下一条指令也会被执行。这种现象是一个空间局部性的例子。提高指令空间局部性的一种有效技术是让编译器把很可能连续执行的多个基本块连续存放（始终按顺序执行的指令序列）即放在同一存储页上，甚至在同一高速缓存线上。属于同一循环或同一函数的指令也很有可能被一起执行。

我们还可以通过改变数据布局或计算顺序来提高程序中数据访问的时间和空间局部性。例如，一些程序重复访问大量数据、而每次访问只完成少量计算，这样的程序性能不佳。如果我们能够将一些数据从存储层次结构的慢速层次加载到较快的层次（例如，从磁盘到主内存），并在该数据驻留在较快层中时执行所有针对该数据的计算，那程序的性能就会变得更好了。这个概念可以递归地应用于物理内存、高速缓存和寄存器中的数据的复用。

The Fragmentation Problem (1)



- At the beginning of program execution, the heap is one contiguous unit of free space
- As the program allocates/deallocates memory, the heap is broken up into free and used chunks



- 在程序执行开始时，堆是一个连续的可用空间单元。
- 随着这个程序分配和回收存储工作的进行，空间被分割成若干空闲存储块和已用存储块，而空闲块不一定位于堆区的某个连续区域中。
- 我们将空闲内存块称为窗口。

The Fragmentation Problem (2)

- With each allocation request, the memory manager must place the requested memory into a **large-enough hole**
 - Typically, it needs to **split** some hole, creating a yet smaller hole
- With each deallocation request, the freed memory are added back to the pool of free space
 - Should **combine contiguous holes** into larger ones. Otherwise, the holes can only get smaller
- Without a good strategy, the free memory may get *fragmented*, consisting of large numbers of small, noncontiguous holes

对于每个分配请求，内存管理器必须将请求的内存块放入足够大的窗口中。除非找到尺寸完全正确的孔，否则我们需要劈开一些孔，创建一个更小的孔。

对于每个回收请求，被回收的内存块都会添加回空闲空间的缓冲池中。我们将连续的窗口合并成更大的窗口，否则窗口只会变得更小。如果我们不小心，空闲内存最终可能会变得碎片化，由大量不连续的小窗口组成。此时，即使可能有足够的总空闲空间，也可能没有足够大的窗口来满足未来的请求。

Object Placement Strategies

- **Best-fit algorithm**

- Allocate the requested memory in the smallest available hole that is large enough
- This algorithm spares the large holes to satisfy subsequent, larger requests, which tends to **improve space utilization**

- **First-fit algorithm**

- An object is placed in the first (lowest-address) hole in which it fits
- It **takes less time** to place objects and **improves spatial locality**, but is inferior to best-fit in overall performance

我们通过控制内存管理器如何在堆中放置新对象来减少碎片。

- 根据经验发现，最小化现实程序碎片的一个好策略是将请求的存储分配在满足请求的最小可用窗口中。这种 best-fit 算法倾向于将大的窗口保留下来以满足后续更大的请求。（提升空间利用率）

- 另一种称为 first-fit 的替代方案，将对象放置到第一个（即地址最低的）能够容纳请求对象的窗口中。这种策略在放置对象时所需的时间较少（提升空间局部性），但在总体性能方面不如 best-fit 性能好。

The Binning Strategy for Best-Fit

- Separate free space chunks into *bins* (容器), according to chunk sizes
- Have more bins for smaller-sized chunks (objects are often small)
- The **Lea memory manager** of the GNU C compiler **gcc** aligns all chunks to 8-byte boundaries
 - There is a bin for every multiple of 8-byte chunks: 16, 24, 32, ..., 512
 - Organizing larger-sized chunks (>512 bytes): the size of the smallest chunk for each bin is twice that of the previous one. Within each bin, the chunks are ordered by size.
 - Wilderness chunk (荒野块): the space can be extended by requesting more pages from the OS. Lea treats this chunk as the largest-sized bin

Learn more about Doug Lea's memory allocator: https://cw.fel.cvut.cz/old/_media/courses/a4m33pal/04_dynamic_memory_v6.pdf

为了更有效地实现 best-fit 放置策略，我们可以根据空闲空间块的大小将它们分在若干个容器中。

一种实用的想法是为较小的尺寸设置较多的容器，因为小对象的个数通常比较多。

例如，GNU C 编译器 gcc 中使用的 Lea 内存管理器将所有块对齐到 8 字节的边界。

- 从 16 字节到 512 字节之间的、每个 8 字节块的整数倍的存储块都设置了一个容器。
- 较大尺寸的容器按照对数间隔（即，每个容器的最小尺寸是前一个 bin 的最小尺寸的两倍）。在每个容器中，块按其大小排序。
- 总有一块可用空间，存储管理器可以向操作系统请求更多页面来扩展这个块。该块称为荒野块，由于其可扩展性，Lea 将其视为最大尺寸存储块的容器。

The Binning Strategy for Best-Fit

- Binning makes it easy to find the best-fit chunk:
 - For **small sizes** requests, if there is a bin for chunks of that size only, we may take any chunk from the bin (the chunks are of the same size)
 - For **sizes that do not have a private bin**, we find the bin that is allowed to include chunks of the size.
 - Within the bin, where the chunks may have different size, we can use either a first-fit or a best-fit algorithm*
 - When the above searches fail, we continue to check the bin for the next larger size(s). Eventually, we may reach **wilderness chunk**
- * When the fit is not exact, the remainder of the chunk will generally need to be placed in a bin with smaller sizes

容器机制使得寻找 best-fit 块变得容易。

- 对于 Lea 内存管理器请求的小尺寸，如果有一个容器仅用于该大小的块，我们可以从该容器中任意取出一个存储块。
- 对于没有专用容器的大小，我们会找到一个允许包含所需大小的块容器。在该容器中，我们可以使用 first-fit 或 best-fit 策略；也就是说，我们要么寻找并选择第一个足够大的块，要么花费更多时间找到足够大的最小块。请注意，如果选择空闲存储块的大小不是正好合适，块的剩余部分通常需要放置在较小尺寸的容器中。
- 然而，目标容器可能是空的，或者该容器中的所有块都太小而无法满足空间请求。在这种情况下，我们只需重复搜索，使用下一个更大尺寸的容器。最终，我们要么找到可以使用的块，要么到达“荒野”块，从这个荒野块中我们肯定可以获得所需的块，但有可能需要请求操作系统为堆区增加更多的内存页。

虽然 best-fit 可以提高空间利用率，但就空间局部性而言，它可能并不是最佳的。程序在同一时间分配的块往往具有相似的访问模式并具有类似的生命周期。因此，将它们靠近放置可以改善程序的空间局部性。best-fit 算法的有用的改进之一是在无法找到精确请求

大小的块的情况下使用另一种对象放置方法。在这种情况下，我们使用 `next-fit` 策略，只要刚分割过的存储块中仍有足够的空间容纳这个对象，我们就把这个对象放置在这个存储块中。`next-fit` 策略还可以提高分配操作的速度。